

TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 7: Divide and Conquer II: Closest Pair, Integer Multiplication, and the FFT

Foreign Trade University

Contents

1	Introduction	1
2	The Closest Pair of Points	1
2.1	Problem statement	1
2.2	The brute-force baseline	2
2.3	A divide-and-conquer algorithm	2
2.4	Correctness: the strip and the neighbor lemma	3
2.5	Running time	3
3	Integer Multiplication: Karatsuba’s Algorithm	4
3.1	The grade-school recurrence	4
3.2	Three multiplications instead of four	4
4	Polynomials and the Convolution Problem	4
4.1	Polynomial multiplication is convolution	4
4.2	Two representations of a polynomial	5
5	The Discrete Fourier Transform and the FFT	5
5.1	Roots of unity	5
5.2	The recursive radix-2 FFT	5
5.3	The inverse FFT	6
5.4	Fast polynomial multiplication via FFT	7
6	Other Divide-and-Conquer Gems	7
7	Summary and Looking Ahead	7

1 Introduction

Last week introduced the **divide-and-conquer** paradigm through mergesort and the master theorem: split an instance into smaller subinstances, solve them recursively, and *combine* the partial answers. The running time of such an algorithm is captured by a recurrence $T(n) = aT(n/b) + (\text{combine cost})$, and the whole art lies in making the combine step cheap enough that the recursion is worthwhile.

This week we push divide-and-conquer in two directions where the combine step is genuinely subtle, and where a naive approach gives a worse bound than a clever one (following Kleinberg & Tardos, Ch. 5):

- **Geometry.** The *closest pair of points* in the plane can be found in $O(n \log n)$ instead of the obvious $O(n^2)$ – but only with a careful “strip” argument that bounds how many points can be packed near the dividing line.
- **Algebra.** Multiplying two n -digit integers, or two degree- n polynomials, looks like an $\Theta(n^2)$ task. *Karatsuba’s* trick brings integer multiplication down to $O(n^{\log_2 3}) \approx O(n^{1.585})$ using three multiplications instead of four, and the *Fast Fourier Transform (FFT)* brings polynomial multiplication down to $O(n \log n)$ by evaluating, multiplying pointwise, and interpolating at carefully chosen complex *roots of unity*.

A recurring lesson: the bottleneck in a divide-and-conquer algorithm is almost never the recursion itself but the combine step, and reducing the *number* of subproblems (from 4 to 3 in Karatsuba, from n evaluations to a recursive halving in the FFT) changes the exponent in the running time.

2 The Closest Pair of Points

2.1 Problem statement

Definition 2.1 (Closest Pair of Points). *Given n points $p_1, \dots, p_n$ in the plane, with $p_i = (x_i, y_i)$, find the pair (p_i, p_j) , $i \neq j$, minimizing the Euclidean distance*

$$d(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}.$$

2.2 The brute-force baseline

The obvious algorithm examines all $\binom{n}{2}$ pairs and keeps the smallest distance.

Algorithm 1 Brute-Force Closest Pair

```

1: best  $\leftarrow +\infty$ 
2: for  $i = 1$  to  $n$  do
3:   for  $j = i + 1$  to  $n$  do
4:     if  $d(p_i, p_j) < \textit{best}$  then
5:       best  $\leftarrow d(p_i, p_j)$ ; remember the pair  $(p_i, p_j)$ 
6:     end if
7:   end for
8: end for
9: return best and the remembered pair

```

This runs in $\Theta(n^2)$ time. It is correct by definition and is the reference implementation that Problem 1 asks you to write, and against which the fast algorithm is checked in Problem 3.

2.3 A divide-and-conquer algorithm

In one dimension the closest pair is trivial: sort the points and scan adjacent pairs in $O(n \log n)$. In two dimensions we cannot simply sort, because closeness in x does not imply closeness in the plane. The idea is to split the points by x -coordinate, recurse, and then carefully handle pairs that straddle the dividing line.

Problem 2 asks you to implement this algorithm so it also returns the achieving pair, not just the distance.

Algorithm 2 Closest-Pair (divide and conquer)

```
1: Sort the points once by  $x$ -coordinate ( $P_x$ ) and once by  $y$ -coordinate ( $P_y$ ).
2: function CLOSESTPAIR( $P_x, P_y$ )
3:   if  $|P_x| \leq 3$  then
4:     return the closest pair by brute force
5:   end if
6:   Let  $L$  be the left half of  $P_x$ ,  $R$  the right half; let  $x^*$  be the median  $x$ .
7:   Split  $P_y$  into  $L_y$  and  $R_y$  (points of  $L$ , resp.  $R$ , in  $y$ -order).
8:    $(\delta_L) \leftarrow \text{CLOSESTPAIR}(L_x, L_y)$ 
9:    $(\delta_R) \leftarrow \text{CLOSESTPAIR}(R_x, R_y)$ 
10:   $\delta \leftarrow \min(\delta_L, \delta_R)$ 
11:  Let  $S$  be the points within horizontal distance  $\delta$  of  $x^*$ , kept in  $y$ -order.
12:  for each point  $s_i$  in  $S$  do
13:    for each of the next 7 points  $s_j$  after  $s_i$  in  $S$  do
14:       $\delta \leftarrow \min(\delta, d(s_i, s_j))$ 
15:    end for
16:  end for
17:  return  $\delta$ 
18: end function
```

2.4 Correctness: the strip and the neighbor lemma

The recursion correctly finds the closest pair *within* the left half and *within* the right half. The only pairs not yet considered are those with one point in L and one in R . If such a “crossing” pair is closer than $\delta = \min(\delta_L, \delta_R)$, both of its points must lie within horizontal distance δ of the dividing line $x = x^*$ – otherwise their x -coordinates alone would differ by more than δ . So only the *strip* S of points with $|x - x^*| < \delta$ can contain a closer crossing pair.

The strip can contain up to all n points, so comparing all pairs in it would not help. The key geometric fact is that, *within the strip*, each point can be close to only a constant number of others.

Lemma 2.2 (Sparsity of the strip). *Let S be the points in the strip, sorted by y -coordinate. If two points $s_i, s_j \in S$ satisfy $d(s_i, s_j) < \delta$, then they are within 7 positions of each other in this sorted order. Equivalently, it suffices for each point to be compared with the next 7 points after it in y -order.*

Proof. Partition the strip (a vertical band of width 2δ centred on $x = x^*$) into a grid of boxes of side $\delta/2$. Each box has diameter $\frac{\delta}{2}\sqrt{2} = \frac{\delta}{\sqrt{2}} < \delta$. Hence *no box can contain two points*: two points in the same box would be at distance $< \delta$, but every two points lying entirely on the same side of x^* are at distance $\geq \delta$ (that side was solved recursively with minimum distance $\geq \delta$), and the box, having width $\delta/2 < \delta$, lies entirely on one side of x^* except for boxes straddling the line; a straddling box of width $\delta/2$ still has diameter $< \delta$, so two points in it would contradict δ being the current minimum.

Now suppose s_i and s_j are within distance δ of each other. Their y -coordinates differ by at most δ , so s_j lies in one of the rows of boxes that are within vertical distance δ of s_i . That region spans the full width of the strip (2δ , i.e. 4 columns of width $\delta/2$) and a height of 2δ (4 rows), giving a 4×4 block of 16 boxes. Since each box holds at most one point, at most 15 other points can be that close, and in y -sorted order any such point is among the next few neighbours. A standard refinement of this counting argument (boxes strictly below or at the same height) shows it suffices

to check the next 7 neighbours; checking 15 is also safe and is sometimes the bound quoted. In all cases the number of comparisons per point is a *constant*. $\square$

Theorem 2.3. *The divide-and-conquer algorithm returns the true closest-pair distance.*

Proof. By induction on n . For $n \leq 3$ the brute-force base case is exact. For larger n , the true closest pair is either entirely in L , entirely in R , or a crossing pair. The first two cases are found by the recursive calls (inductive hypothesis). A crossing pair closer than δ must have both endpoints in the strip S , and by Lemma 2.2 the algorithm compares every pair of strip points that could be within δ (each point to its next 7 neighbours in y -order). Hence the algorithm examines the true closest pair in every case and returns its distance. $\square$

2.5 Running time

Sorting by x and y is a one-time $O(n \log n)$ cost. Each recursive call splits P_y in linear time and does $O(|S|) = O(n)$ work in the strip (a constant number of comparisons per strip point). The recurrence is therefore

$$T(n) = 2T(n/2) + O(n) = O(n \log n),$$

matching mergesort. The whole algorithm is $O(n \log n)$, a substantial improvement over the $\Theta(n^2)$ brute force. Problem 3 verifies the two agree on random point sets.

3 Integer Multiplication: Karatsuba's Algorithm

3.1 The grade-school recurrence

Write two n -digit numbers (in some base B , e.g. $B = 10$ or $B = 2$) by splitting each into a high and low half of $n/2$ digits:

$$x = x_1 B^{n/2} + x_0, \quad y = y_1 B^{n/2} + y_0.$$

Then

$$xy = x_1 y_1 B^n + (x_1 y_0 + x_0 y_1) B^{n/2} + x_0 y_0.$$

Computing the four products $x_1 y_1$, $x_1 y_0$, $x_0 y_1$, $x_0 y_0$ recursively gives

$$T(n) = 4T(n/2) + O(n) = \Theta(n^2),$$

by the master theorem ($\log_2 4 = 2$). This is no better than the grade-school algorithm.

3.2 Three multiplications instead of four

Karatsuba's observation: the middle coefficient $x_1 y_0 + x_0 y_1$ can be recovered from *one* extra product rather than two. Compute

$$z_2 = x_1 y_1, \quad z_0 = x_0 y_0, \quad z_1 = (x_0 + x_1)(y_0 + y_1) - z_2 - z_0,$$

and note $z_1 = x_1 y_0 + x_0 y_1$. Then $xy = z_2 B^n + z_1 B^{n/2} + z_0$. This uses only **three** recursive multiplications (z_0 , z_2 , and the product inside z_1), plus additions and shifts costing $O(n)$.

Theorem 3.1. *Karatsuba's algorithm multiplies two n -digit integers in time*

$$T(n) = 3T(n/2) + O(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585}).$$

Proof. The combine work (three additions/subtractions of $O(n)$ -digit numbers and constant-factor shifts) is $O(n)$. With $a = 3$, $b = 2$, and combine exponent $d = 1$, we have $a = 3 > b^d = 2$, so by the master theorem $T(n) = \Theta(n^{\log_b a}) = \Theta(n^{\log_2 3})$. $\square$

Problem 4 asks you to implement Karatsuba with exactly three recursive multiplications and to count base-case multiplications with the shared **Counter** helper. The same divide-and-multiply idea reappears in *Strassen's* matrix multiplication, which multiplies two $n \times n$ matrices with 7 recursive multiplications of $(n/2) \times (n/2)$ blocks instead of the obvious 8, giving $T(n) = 7T(n/2) + O(n^2) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$ – the subject of Problem 9.

4 Polynomials and the Convolution Problem

4.1 Polynomial multiplication is convolution

Represent a polynomial $A(x) = \sum_{i=0}^{n-1} a_i x^i$ by its coefficient vector $(a_0, \dots, a_{n-1})$. The product $C(x) = A(x)B(x)$ has coefficients

$$c_k = \sum_{i+j=k} a_i b_j, \quad k = 0, 1, \dots, (\deg A + \deg B),$$

which is exactly the (linear) **convolution** of the two coefficient vectors. The straightforward double loop computes all c_k in $\Theta(nm)$ time (Problem 5). Convolution is ubiquitous: it is also what you compute when multiplying big integers (digits are coefficients), filtering a signal, or cross-correlating two sequences (Problem 8).

4.2 Two representations of a polynomial

A degree- $(n-1)$ polynomial is determined by either

- (a) its **coefficients** $(a_0, \dots, a_{n-1})$, or
- (b) its **values** at n distinct points $A(x_0), \dots, A(x_{n-1})$ (Lagrange interpolation: n points determine a unique degree- $< n$ polynomial).

The crucial asymmetry: *multiplication is cheap in the value representation*. If we know A and B at the same $2n$ points, then $C = AB$ at those points is just the $2n$ pointwise products $C(x_k) = A(x_k)B(x_k)$ – linear time. The challenge is converting between the two representations quickly, i.e. *evaluating* a polynomial at many points and *interpolating* back. Done naively each is $\Theta(n^2)$. The FFT makes both $O(n \log n)$ by choosing the evaluation points cleverly.

5 The Discrete Fourier Transform and the FFT

5.1 Roots of unity

The clever choice of evaluation points is the n -th **roots of unity**. Let

$$\omega_n = e^{2\pi i/n},$$

so $\omega_n^0, \omega_n^1, \dots, \omega_n^{n-1}$ are the n distinct complex numbers with $z^n = 1$, equally spaced on the unit circle. They satisfy two properties that drive the whole algorithm:

- **Squaring (halving) property.** $(\omega_n^k)^2 = \omega_{n/2}^k$ – squaring the n -th roots gives the $(n/2)$ -th roots, each appearing twice. This is what lets the problem split in half.
- **Cancellation property.** $\omega_n^{k+n/2} = -\omega_n^k$, so a root and its “antipode” differ only in sign.

Definition 5.1 (Discrete Fourier Transform). *The DFT of a coefficient vector $(a_0, \dots, a_{n-1})$ is the vector of values*

$$\hat{a}_k = A(\omega_n^k) = \sum_{j=0}^{n-1} a_j \omega_n^{jk}, \quad k = 0, \dots, n-1.$$

This is evaluation of A at all n roots of unity simultaneously.

Computing the DFT by the definition costs $\Theta(n^2)$. The FFT computes it in $O(n \log n)$.

5.2 The recursive radix-2 FFT

Assume n is a power of two (pad with zero coefficients otherwise – Problem 6). Split A into its even- and odd-indexed coefficients:

$$A_{\text{even}}(y) = \sum_j a_{2j} y^j, \quad A_{\text{odd}}(y) = \sum_j a_{2j+1} y^j,$$

each of degree $< n/2$. Then

$$A(x) = A_{\text{even}}(x^2) + x A_{\text{odd}}(x^2).$$

Evaluate at $x = \omega_n^k$. By the squaring property, $(\omega_n^k)^2 = \omega_{n/2}^k$, so A_{even} and A_{odd} need only be evaluated at the $(n/2)$ -th roots of unity – which are themselves two recursive DFTs of half the size. Using the cancellation property to pair k with $k + n/2$:

$$A(\omega_n^k) = A_{\text{even}}(\omega_{n/2}^k) + \omega_n^k A_{\text{odd}}(\omega_{n/2}^k), \quad A(\omega_n^{k+n/2}) = A_{\text{even}}(\omega_{n/2}^k) - \omega_n^k A_{\text{odd}}(\omega_{n/2}^k),$$

for $k = 0, \dots, n/2 - 1$. Each output pair is computed from one even-DFT value and one odd-DFT value (the “butterfly”).

Algorithm 3 Recursive Radix-2 FFT

```

1: function FFT( $(a_0, \dots, a_{n-1})$ )
2:   if  $n = 1$  then return  $(a_0)$ 
3:   end if
4:    $E \leftarrow \text{FFT}((a_0, a_2, a_4, \dots))$  ▷ even indices
5:    $O \leftarrow \text{FFT}((a_1, a_3, a_5, \dots))$  ▷ odd indices
6:   for  $k = 0$  to  $n/2 - 1$  do
7:      $t \leftarrow \omega_n^k O[k]$  ▷  $\omega_n = e^{-2\pi i/n}$  for the forward transform
8:      $\hat{a}[k] \leftarrow E[k] + t$ 
9:      $\hat{a}[k + n/2] \leftarrow E[k] - t$ 
10:  end for
11:  return  $(\hat{a}_0, \dots, \hat{a}_{n-1})$ 
12: end function

```

Theorem 5.2. *The recursive radix-2 FFT computes the DFT of a length- n vector (with n a power of two) in time $T(n) = 2T(n/2) + O(n) = O(n \log n)$.*

Proof. Two recursive calls of size $n/2$, plus $O(n)$ work for the $n/2$ butterflies, give the mergesort recurrence $T(n) = 2T(n/2) + O(n) = O(n \log n)$. Correctness is the even/odd identity above, applied at the roots of unity using their squaring and cancellation properties. $\square$

5.3 The inverse FFT

To return from the value representation to coefficients we need the inverse DFT. Written as a matrix, the DFT is multiplication by the Vandermonde matrix V with $V_{kj} = \omega_n^{kj}$. Its inverse is remarkably simple:

$$(V^{-1})_{kj} = \frac{1}{n} \omega_n^{-kj}.$$

That is, the inverse DFT has the *same form* as the forward DFT but with ω_n replaced by $\omega_n^{-1} = e^{-2\pi i/n}$ (or $e^{+2\pi i/n}$, depending on the sign convention for the forward transform), followed by dividing every entry by n .

Lemma 5.3 (Inversion). $VV^{-1} = I$; equivalently, $\frac{1}{n} \sum_{k=0}^{n-1} \omega_n^{k(j-j')} = 1$ if $j = j'$ and 0 otherwise.

Proof. If $j = j'$ every term is $\omega_n^0 = 1$, summing to n , so $\frac{1}{n} \cdot n = 1$. If $j \neq j'$, let $r = \omega_n^{j-j'} \neq 1$; then $\sum_{k=0}^{n-1} r^k = \frac{r^n - 1}{r - 1} = 0$ since $r^n = (\omega_n^n)^{j-j'} = 1$. $\square$

So the inverse FFT is just the forward FFT run with the conjugate root, divided by n – Problem 6 asks you to implement both `fft` and `ifft` and check that `ifft(fft(x)) == x`.

5.4 Fast polynomial multiplication via FFT

Putting the pieces together gives the $O(n \log n)$ multiplication algorithm:

Algorithm 4 Polynomial Multiplication via FFT

- 1: Let m be a power of two with $m \geq (\deg A + \deg B + 1)$; zero-pad A and B to length m .
 - 2: $\hat{A} \leftarrow \text{FFT}(A)$, $\hat{B} \leftarrow \text{FFT}(B)$ $\triangleright$ evaluate at the m -th roots of unity
 - 3: $\hat{C}[k] \leftarrow \hat{A}[k] \cdot \hat{B}[k]$ for all k $\triangleright$ pointwise product, $O(m)$
 - 4: $C \leftarrow \text{IFFT}(\hat{C})$ $\triangleright$ interpolate back to coefficients
 - 5: Round C to the nearest integers (when inputs are integer-valued) and return.
-

Theorem 5.4. *Two polynomials of degree $< n$ can be multiplied in $O(n \log n)$ arithmetic operations via the FFT.*

Proof. The two forward FFTs and one inverse FFT each cost $O(m \log m)$ with $m = O(n)$, and the pointwise multiplication costs $O(m)$. The total is $O(n \log n)$. Correctness: $\hat{C}[k] = \hat{A}[k] \hat{B}[k] = A(\omega^k) B(\omega^k) = C(\omega^k)$, so $\hat{C}$ is the value representation of the true product $C = AB$ at the m roots of unity; since $\deg C < m$, interpolation (the inverse FFT) recovers C exactly. $\square$

Problem 7 asks you to implement this and confirm it matches the naive convolution (after rounding to integers). Because integer multiplication is convolution of digits, the same FFT machinery underlies the fastest known big-integer multiplication algorithms; convolution and cross-correlation of integer sequences (Problem 8) are immediate applications.

6 Other Divide-and-Conquer Gems

This week's exercises also revisit divide-and-conquer in three further settings:

- **Counting (range sums).** The number of contiguous subarrays whose sum lies in $[\ell, u]$ can be counted in $O(n \log n)$ by a mergesort over the prefix sums, with a two-pointer count during the merge (Problem 10) – the same “count while you merge” pattern as counting inversions in Week 6.
- **Median of two sorted arrays.** A binary search that partitions the shorter array finds the median of the combined data in $O(\log(m + n))$ (Problem 11), a textbook “divide by halving the search” algorithm.
- **Fast exponentiation.** Square-and-multiply computes $a^e \bmod m$ with $O(\log e)$ multiplications by recursively halving the exponent (Problem 12) – the same halving idea as the FFT, applied to exponents.

7 Summary and Looking Ahead

This week extended divide and conquer to problems whose “combine” step is the real challenge:

- **Closest pair of points:** $O(n \log n)$ via splitting on x , recursing, and a strip scan justified by the sparsity (7/15-neighbour) lemma.
- **Karatsuba multiplication:** three half-size multiplications instead of four, giving $\Theta(n^{\log_2 3})$; Strassen does the analogous trick for matrices (7 vs 8).
- **Polynomials, convolution, and the FFT:** evaluate at the roots of unity (forward FFT), multiply pointwise, interpolate (inverse FFT), for $O(n \log n)$ polynomial and big-integer multiplication.

The unifying theme is that *reducing the number of subproblems, or choosing the right representation, changes the exponent in the recurrence*. Karatsuba and Strassen shrink the branching factor; the FFT changes the problem into a representation (values at roots of unity) where the expensive operation becomes trivial.

Next week we turn to **dynamic programming** (Kleinberg & Tardos, Ch. 6). Where divide and conquer splits a problem into *independent* subproblems, dynamic programming tackles problems whose subproblems *overlap*: we will define recurrences over a polynomial-sized table of subproblems and fill it bottom-up, starting with weighted interval scheduling and the knapsack problem.